



SERVICIO NACIONAL DE APRENDIZAJE - SENA

DISEÑO Y DESARROLLO DE SERVICIOS WEB - CASO

GA7-220501096-AA5-EV01

Proyecto SAAAF
Servicio web de registro e inicio de sesión

Aprendiz	Brayan Giraldo García
Programa	Análisis y Desarrollo de Software
Proyecto formativo	Construcción de software integrador de tecnologías orientadas a servicios
Fase	Ejecución
Repositorio	https://github.com/Brayan-Giraldo/SAAAF

1. Introducción

La evidencia GA7-220501096-AA5-EV01 solicita diseñar y codificar un servicio web que permita registrar un usuario y posteriormente validar un inicio de sesión mediante usuario y contraseña. El servicio debe responder satisfactoriamente cuando las credenciales son correctas y retornar un error de autenticación cuando no coinciden. Para contextualizar el caso, la solución se integra al proyecto SAAAF (Sistema de Asignación de Actas de Activos Fijos) como un servicio de autenticación independiente y reutilizable.

2. Objetivos

2.1 Objetivo general

Diseñar, implementar y verificar un servicio web reutilizable de registro e inicio de sesión para SAAAF, aplicando validaciones, comentarios de código y control de versiones.

2.2 Objetivos específicos

- Implementar un endpoint para registrar usuarios.
- Implementar un endpoint para validar el inicio de sesión.
- Retornar mensajes y códigos HTTP coherentes para autenticación correcta e incorrecta.
- Aplicar validaciones de formato, longitud y duplicidad.
- Mantener la contraseña protegida mediante hash y salt.
- Preparar el proyecto para versionamiento con Git y publicación en GitHub.

3. Alcance y arquitectura

El módulo se desarrolló con Node.js utilizando únicamente módulos estándar. La API expone dos servicios de autenticación y una ruta de salud. Como apoyo para la demostración, se incluye una interfaz web sencilla que consume los mismos endpoints mediante fetch. La información de usuarios se conserva en un archivo JSON local; la contraseña no se almacena en texto plano.

Capa	Archivo	Responsabilidad
Interfaz	public/index.html / app.js	Formulario de registro e inicio de sesión.
API	src/server.js	Recibe solicitudes HTTP y entrega respuestas JSON.
Dominio	src/authStore.js	Valida credenciales, registra usuarios y autentica.
Persistencia	data/usuarios.json	Almacena usuarios con salt y passwordHash.

4. Servicios web implementados

Método	Endpoint	Uso	Estados
GET	/api/health	Verificar disponibilidad	200
POST	/api/auth/register	Registrar usuario	201 / 400 / 409
POST	/api/auth/login	Iniciar sesión	200 / 400 / 401

4.1 Servicio de registro

El registro recibe las propiedades usuario y contraseña. Si ambos datos son válidos y el usuario no existe, se crea el registro y se responde HTTP 201. Si el usuario ya está registrado, se devuelve HTTP 409.

```
POST /api/auth/register
{"usuario":"bgiraldo","contrasena":"Saaaf2026!"}
```

```
HTTP 201
{"message":"Usuario registrado correctamente","usuario":{"id":1,"usuario":"bgiraldo"}}
```

4.2 Servicio de inicio de sesión

El inicio de sesión consulta el usuario registrado y compara la contraseña recibida con el hash almacenado. Cuando coincide, responde HTTP 200 con un mensaje de autenticación satisfactoria. Si el usuario no existe o la contraseña es incorrecta, responde HTTP 401 con un mensaje genérico de error de autenticación.

```
POST /api/auth/login
{"usuario":"bgiraldo","contrasena":"Saaaf2026!"}
```

```
HTTP 200
{"message":"Autenticacion satisfactoria","usuario":{"id":1,"usuario":"bgiraldo"}}
```

```
HTTP 401
{"error":"Error en la autenticacion"}
```

5. Validaciones de verificación

Validación	Comportamiento	HTTP
Campos obligatorios	Rechaza solicitudes sin usuario o contraseña.	400
Longitud de usuario	Exige entre 4 y 40 caracteres.	400
Formato de usuario	Permite minúsculas, números, punto, guion y guion bajo.	400
Longitud de contraseña	Exige entre 8 y 72 caracteres.	400
Usuario duplicado	Evita registrar dos veces el mismo usuario.	409
Credenciales incorrectas	Retorna error genérico de autenticación.	401

Para reforzar la seguridad del ejercicio, las contraseñas se protegen con scrypt y un salt aleatorio. Esta implementación evita conservar la contraseña original en el archivo de datos.

6. Comentarios y estándar de codificación

El código fuente incluye comentarios sobre las responsabilidades principales de autenticación y las rutas solicitadas. Se aplican nombres descriptivos, separación de responsabilidades y funciones de propósito único. El proyecto utiliza módulos ES y no requiere dependencias externas.

```
// Rutas principales comentadas en src/server.js
POST /api/auth/register -> store.register(...)
POST /api/auth/login -> store.login(...)
```

7. Pruebas realizadas

Se incorporó un conjunto automático de pruebas que inicia el servicio en un puerto temporal y verifica las respuestas de registro, autenticación y validaciones. La ejecución realizada durante la preparación de la evidencia obtuvo 10 de 10 pruebas aprobadas.

Prueba	Resultado esperado	Estado
Health responde correctamente	200	Aprobada
Registro de usuario válido	201	Aprobada
Confirmación del usuario registrado	Datos correctos	Aprobada
Registro de usuario duplicado	409	Aprobada
Inicio de sesión correcto	200	Aprobada
Mensaje de autenticación satisfactoria	Mensaje correcto	Aprobada
Contraseña incorrecta	401	Aprobada
Mensaje de error de autenticación	Mensaje correcto	Aprobada
Usuario inexistente	401	Aprobada
Datos inválidos	400	Aprobada

8. Versionamiento

El proyecto fue inicializado con Git y se generó un commit local de la evidencia. Se incluyen .gitignore, VERSIONAMIENTO.md, historial_git.txt y enlace_repositorio.txt. La ruta recomendada para publicar esta evidencia en el repositorio SAAAF es GA7-220501096-AA5-EV01/.

```
git add .
git commit -m "Implementa servicio de registro e inicio de sesion - GA7-AA5-EV01"
git push
```

9. Correspondencia con la lista de chequeo

Indicador	Evidencia desarrollada	Soporte
1. Realiza un servicio para ser utilizado en un registro.	POST /api/auth/register y formulario web de registro.	Sí
2. Realiza un servicio para ser utilizado en un inicio de sesión.	POST /api/auth/login y formulario web de acceso.	Sí
3. Realiza las validaciones de verificación correctamente.	Campos, longitudes, duplicados y credenciales incorrectas.	Sí
4. Utiliza herramientas de versionamiento para la creación del proyecto.	Git local, .gitignore, historial y enlace a GitHub.	Sí

10. Instrucciones de ejecución

1. Abrir PowerShell o una terminal en 01_Proyecto_Servicio_Web_SAAAF.
2. Ejecutar npm.cmd start. Si PowerShell bloquea npm.ps1, ejecutar node .\src\server.js.
3. Abrir <http://localhost:3001/> para probar el registro y el inicio de sesión.
4. Opcionalmente ejecutar npm.cmd test para comprobar las validaciones automáticas.

11. Conclusiones

La solución desarrollada cumple el caso solicitado por la actividad: dispone de un servicio de registro, un servicio de inicio de sesión, respuestas diferenciadas para autenticación correcta e incorrecta, validaciones y comentarios de código. El módulo se diseñó como un componente reutilizable dentro de SAAAF y quedó preparado para su control de versiones y publicación en GitHub.